Systemy operacyjne – Laboratorium 7

Programowanie systemowe w systemie Linux - Procesy

Należy wykonać zadanie/zadania (każde ma inny poziom trudności i pozwala uzyskać inną ocenę maksymalną). Wykonanie zadania na wyższą ocenę jest możliwe dopiero po wykonaniu zadania na ocenę niższą.

Dla każdego zadania, proszę przygotować odpowiedni kod programu, opatrzony niezbędnymi komentarzami, wyjaśniającymi najważniejsze fragmenty kodu (z wyjątkiem zadania 3).

Pliki z kodami powinny być umieszczone w folderach nazwanych odpowiednio: ***zad1, zad2, zad3***.

Wskazane foldery powinny być zawarte w archiwum ***tar.gz*** o nazwie: ***SO_inf1s_L0X_lab7_Nazwisko_Imie.tar.gz***

Kody proszę przesłać przez maila na adres p.ziemba@pm.szczecin.pl

Temat wiadomości: SO_inf1s_L0X_lab7_Nazwisko_Imie

Zmodyfikuj odpowiednio nazwę pliku i temat zależnie od grupy, do której należysz i numeru laboratorium! X to numer grupy - dopasuj go zależnie od grupy, do której należysz: 1 lub 2

Niewysłanie wiadomości, wysłanie jej w formie niezgodnej z powyższymi wymaganiami lub wysłanie pliku/plików, który nie będzie się poprawnie kompilował i uruchamiał, będzie traktowane jako brak programu i skutkowało otrzymaniem za niego oceny niedostatecznej.

Zadanie 1. (na ocenę 3.0)

Napisać w języku C program spełniający poniższe wymagania.

- Program jest uruchamiany z dwoma argumentami wejściowymi – liczbami całkowitymi, których wartości są uporządkowane rosnąco (arg1 < arg2).
    - Wywołanie programu z inną liczbą argumentów albo bez argumentów, powinno skutkować wypisaniem na stdout informacji o przyczynie błędnego wywołania („Błędna liczba argumentów”) i zakończeniem pracy z kodem zakończenia równym -100.
    - Wywołanie programu z dwoma argumentami, ale z niepoprawnie uporządkowanymi wartościami (arg1 ≥ arg2), powinno skutkować wypisaniem na stdout informacji o przyczynie błędnego wywołania („Niepoprawne uporządkowanie argumentów”) i zakończeniem pracy z kodem zakończenia równym -200.
- Jeżeli liczba argumentów i ich uporządkowanie jest poprawne program wykonuje się, uruchamiając proces potomny (funkcja fork()).
- Następnie każdy z procesów (rodzic i potomek) wypisuje na stdout odpowiednią informację identyfikacyjną.
    - Rodzic wypisuje „Rodzic mowi halo” – liczba wypisań tej informacji ma być równa wartości pierwszego argumentu wywołania programu.
    - Potomek wypisuje „Dziecko mowi czesc” – liczba wypisań tej informacji ma być równa wartości drugiego argumentu wywołania programu. Następnie potomek kończy działanie z kodem 22.
- Rodzic czeka na zakończenie procesu potomnego, a następnie wypisuje na stdout PID zakończonego potomka i jego kod zakończenia. Następnie rodzic kończy pracę.

Przykładowe wywołanie i wynik wykonania programu:

```
$ ./lab7zad1 2 4
Dziecko mowi czesc
Rodzic mowi halo
Dziecko mowi czesc
Rodzic mowi halo
Dziecko mowi czesc
```

```
Dziecko mowi czesc
Dziecko STOP: PID = 1923
Dziecko EXIT: CODE = 22
```

Zadanie 2. (na ocenę 4.0)
Zrealizować wyszukiwanie liczby maksymalnej i minimalnej w tablicy liczb całkowitych.
Program napisany w języku C powinien spełniać poniższe wymagania.

- Program jest uruchamiany z *n* argumentami wejściowymi (liczbami całkowitymi), przy czym $20 \geq n \geq 1$.
    - Wywołanie programu z inną liczbą argumentów albo bez argumentów, powinno skutkować wypisaniem na stdout informacji o przyczynie błędnego wywołania („Błędna liczba argumentów") i zakończeniem pracy z kodem zakończenia równym -100.
- Jeżeli liczba argumentów jest poprawna program wykonuje się dalej.
    - Jeżeli liczba argumentów wynosi 2 lub 1 - wówczas wartość elementu maksymalnego i minimalnego wyznaczana jest w jednym porównaniu lub bez porównania (gdy rozmiar wynosi 1).
    - W przeciwnym przypadku ciąg *n* argumentów dzielony jest na dwie „połowy" (jeśli długość łańcucha nie jest parzysta, druga z „połówek" ma być dłuższa o jeden znak). Następnie program uruchamia cztery procesy potomne i każdemu z potomków przekazuje:
        - dla pierwszego i trzeciego dziecka – pierwszą połowę łańcucha argumentów,
        - dla drugiego i czwartego dziecka – drugą połowę łańcucha argumentów.
    - Potomek pierwszy i drugi poszukują maksimum w zadanym im fragmencie łańcucha argumentów.
    - Potomek trzeci i czwarty poszukują minimum w zadanym im fragmencie łańcucha argumentów.
    - Każdy z potomków zwraca rodzicowi informację o ekstremalnej (maksymalnej lub minimalnej) wartości w swojej części łańcucha, a także wypisuje na ekranie: pid rodzica, swój pid, badany fragment łańcucha argumentów, informację, czy szuka max czy min, i ustaloną wartość ekstremalną.
    - Rodzic czeka na zakończenie potomków, porównuje wartości zwrócone przez potomków szukających max i wybiera większą, wypisując ją na ekranie jako max. Porównuje także wartości zwrócone przez potomków szukających min i wybiera mniejszą, wypisując ją na ekranie jako min.

Przykładowe wywołanie i wynik wykonania programu:

```
$ ./lab7zad2 2 4 7 6
1614 1734 2 4 max 4
1614 1735 7 6 max 7
1614 1741 2 4 min 2
1614 1743 7 6 min 6
max 7
min 2
```

Zadanie 3. (na ocenę 5.0)
Podziel kod programu z zadania 2 na pliki nagłówkowe h i pliki c. Pik nagłówkowy funkcje.h ma zawierać nagłówki autorskich funkcji, wykorzystanych w programie (np. funkcje porównania). Każda autorska funkcja ma być zapisana w osobnym pliku, nazwanym tak jak funkcja.

Skonstruuj plik makefile, który będzie odpowiadał za kompilację kodu i dodatkowo sprzątał pozostałości po kompilacji (usuwał zbędne pliki tworzone podczas kompilacji).
Kod ma kompilować się wywołaniem *make*.

Funkcje, które mogą przydać się do zadań 1-2:
fork(),
exec…()
wait(),
waitpid(),
getpid(),
getppid(),
exit(),
makra WIFEXITED(), WEXITSTATUS()